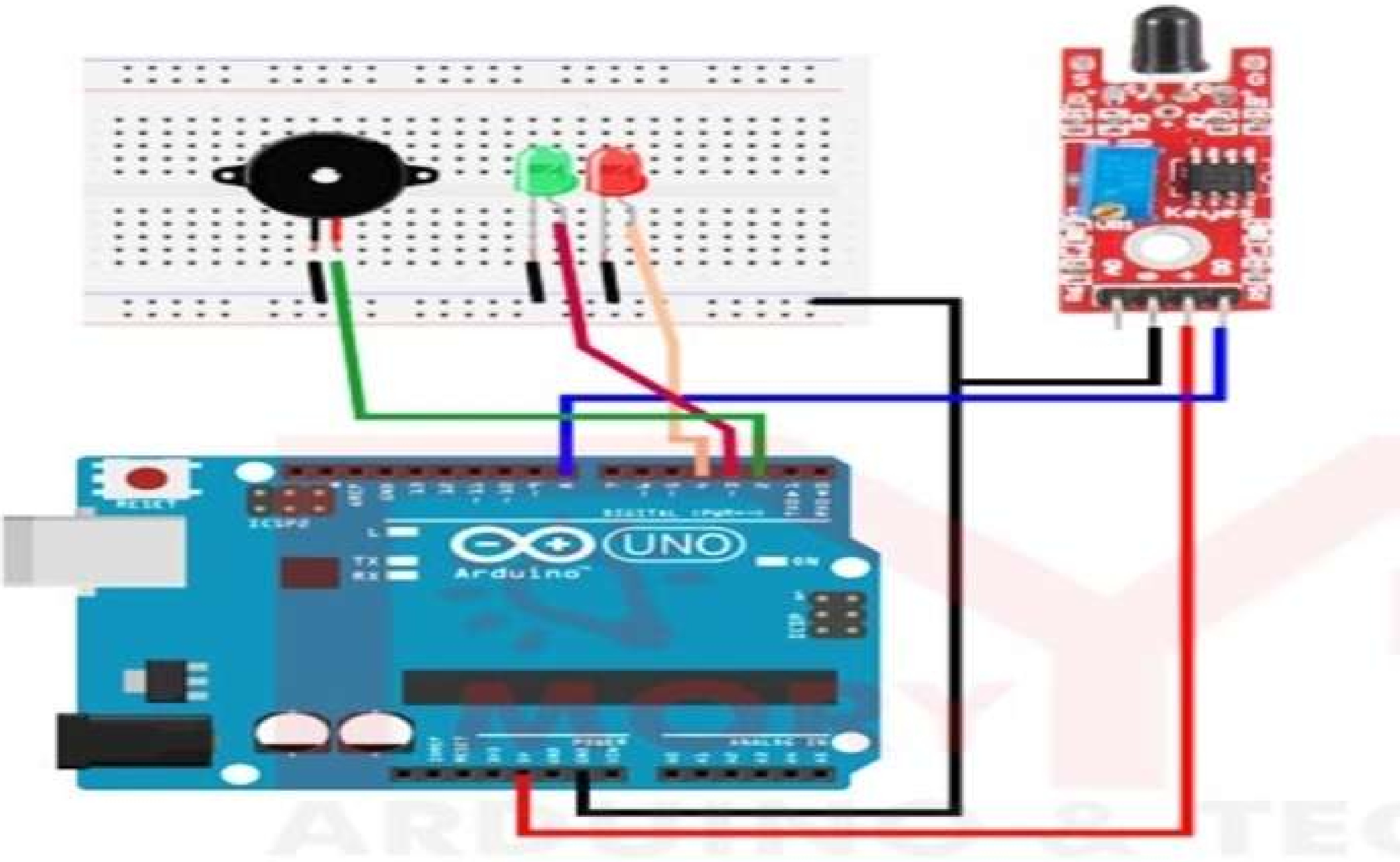
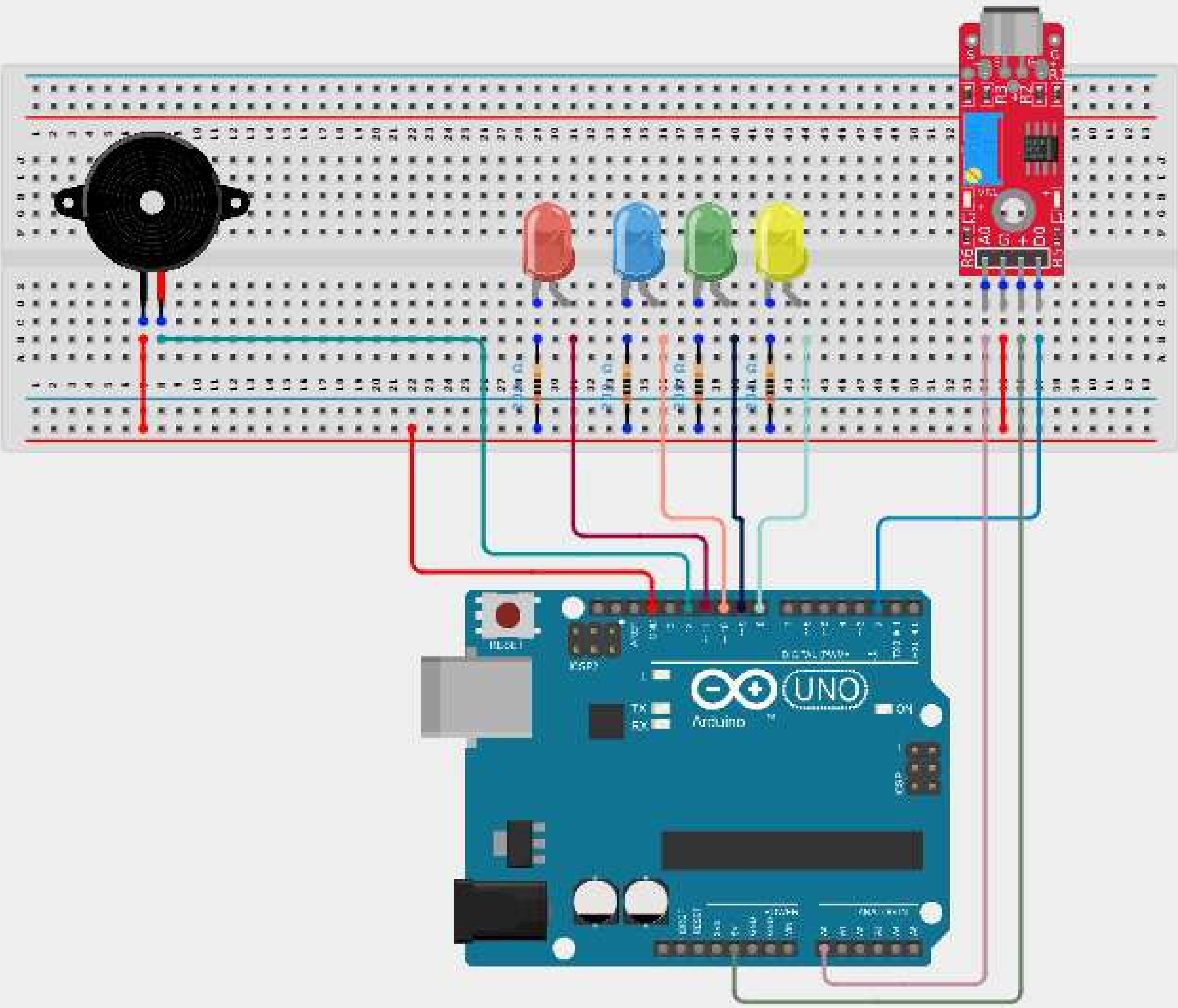

UNO
Arduino


RESET
UNO
Arduino
TX
RX
ON
DIGITAL (PWM~)
A0
G
+
D0
R6
R3
R2

Getting to Know Your DHT Sensor

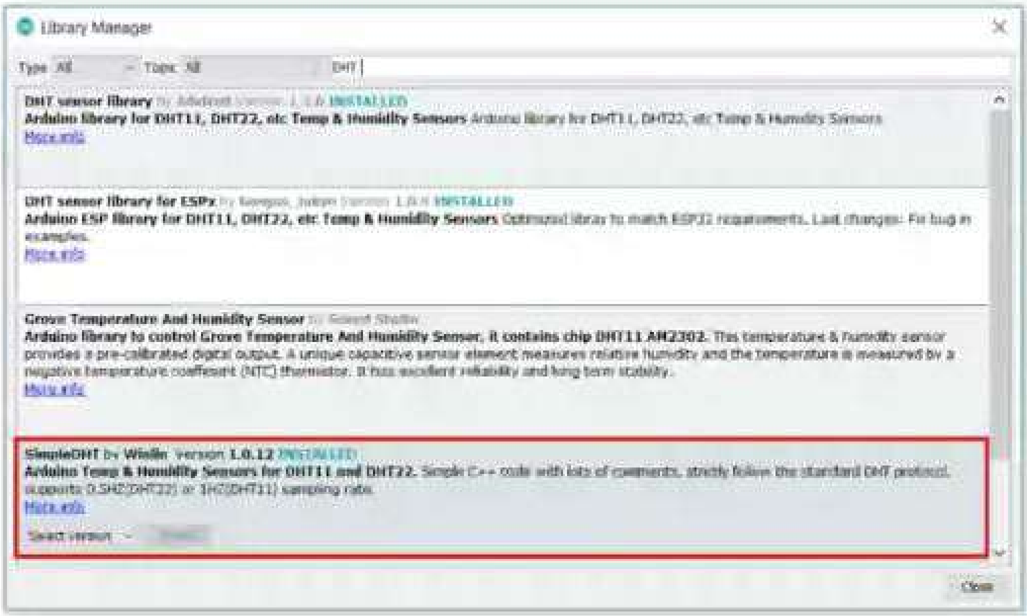
"The DHT11 is a basic, low-cost digital temperature and humidity sensor. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air.

It's fairly simple to use but requires careful timing to grab data.

The only real downside of this sensor is you can only get new data from it once every 2 seconds. "

Getting to Know Your DHT Sensor

Installing Library:



Getting to Know Your DHT Sensor

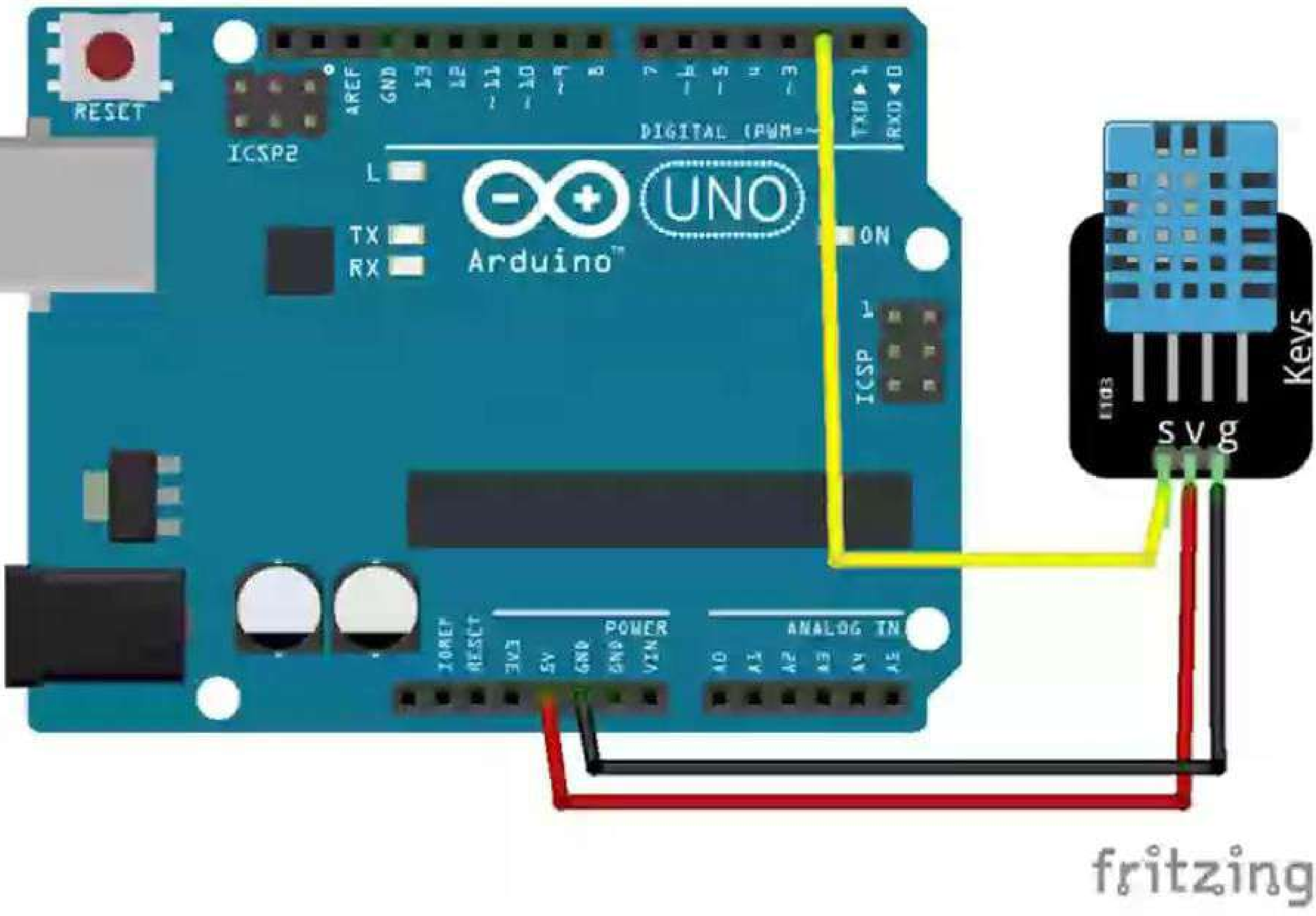
<u>Library Codes:</u>

Code	Code Uses	Code terms
SimpleDHT11 ht11(pinDHT11);	This code is used to define the pin type which has the sensor connected to it.	SimpleDHT11 ht11 (pinDHT11) ; Comment Sensor Name Pin number
dht11.read (&temp, &h, NULL);	This code is used to read the values from the sensor and store them in the specified variables. In our case, they are the "temp" and "h". The "&" symbol is required in the code.	dht11.read(&temp, &h, NULL); Sensor name Order Humidity Value Temperature value Code requirement

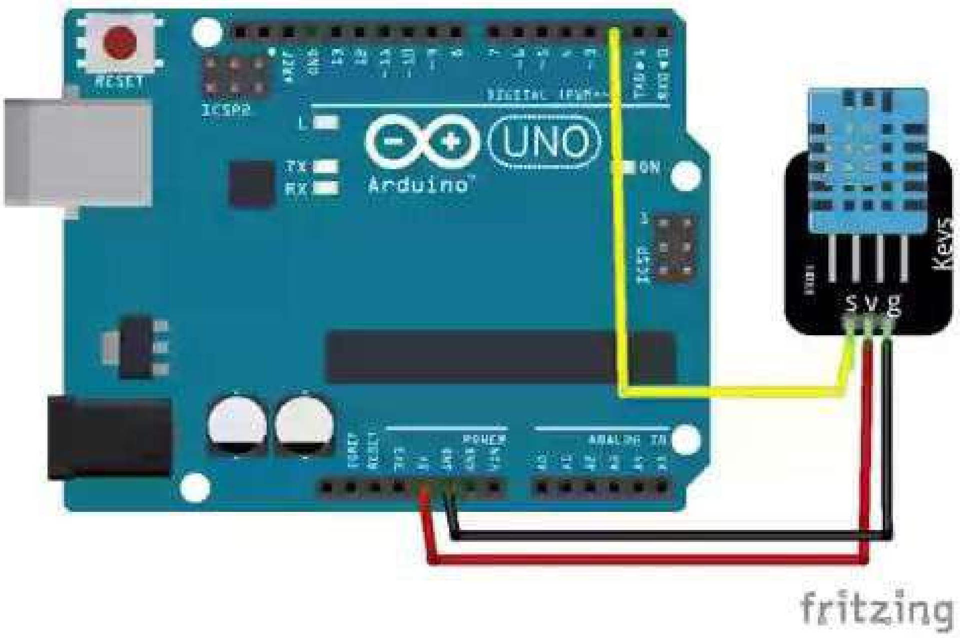
Getting to Know Your DHT Sensor

<u>Library Codes:</u>

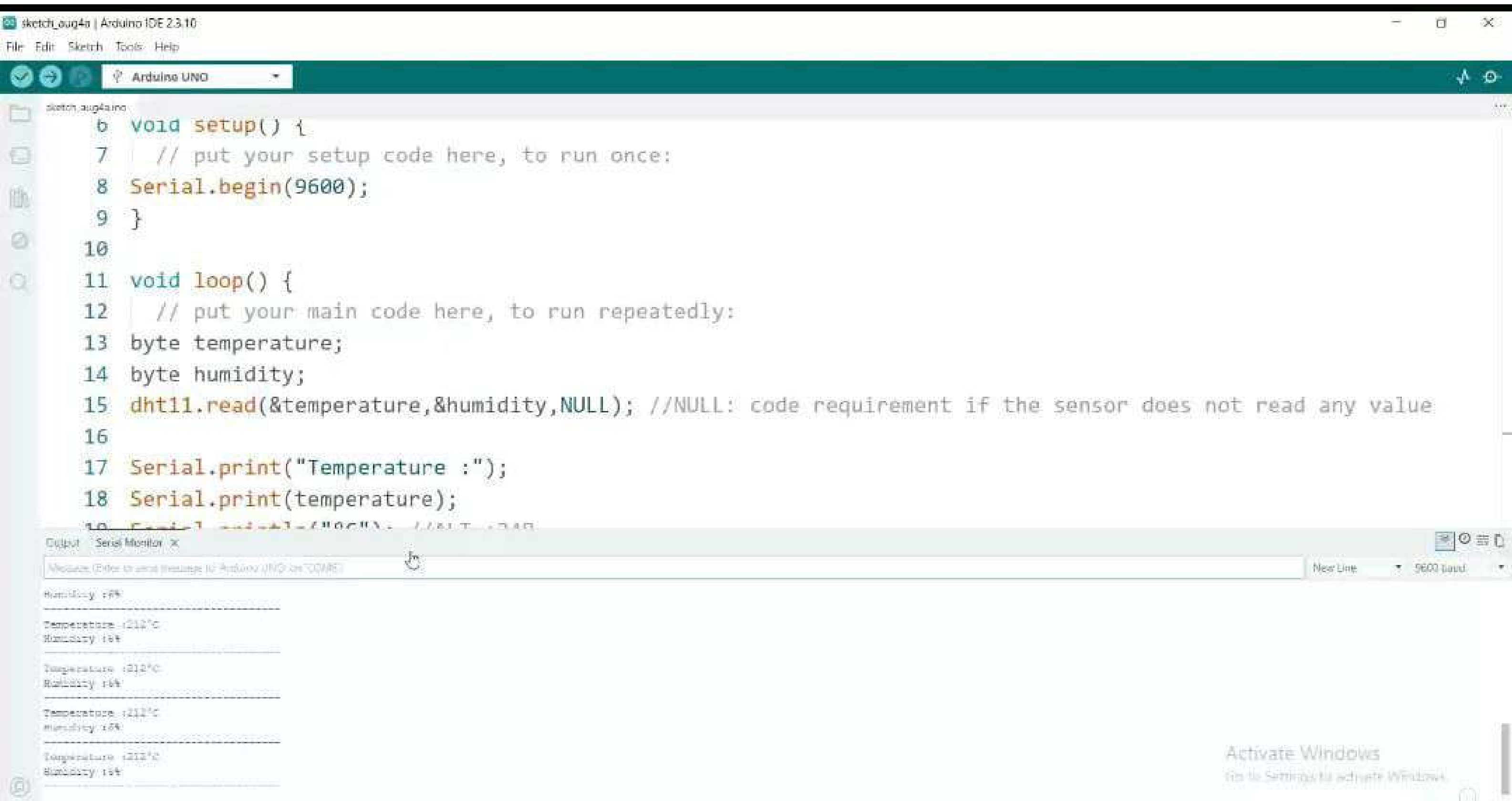
Code	Code Uses	Code terms
SimpleDHT11 ht11(pinDHT11);	This code is used to define the pin type which has the sensor connected to it.	SimpleDHT11 ht11 (pinDHT11) ; Comment Sensor Name Pin number
dht11.read (&temp, &h, NULL);	This code is used to read the values from the sensor and store them in the specified variables. In our case, they are the "temp" and "h". The "&" symbol is required in the code.	dht11.read(&temp, &h, NULL); Sensor name Order Temperature value Humidity Value Code requirement

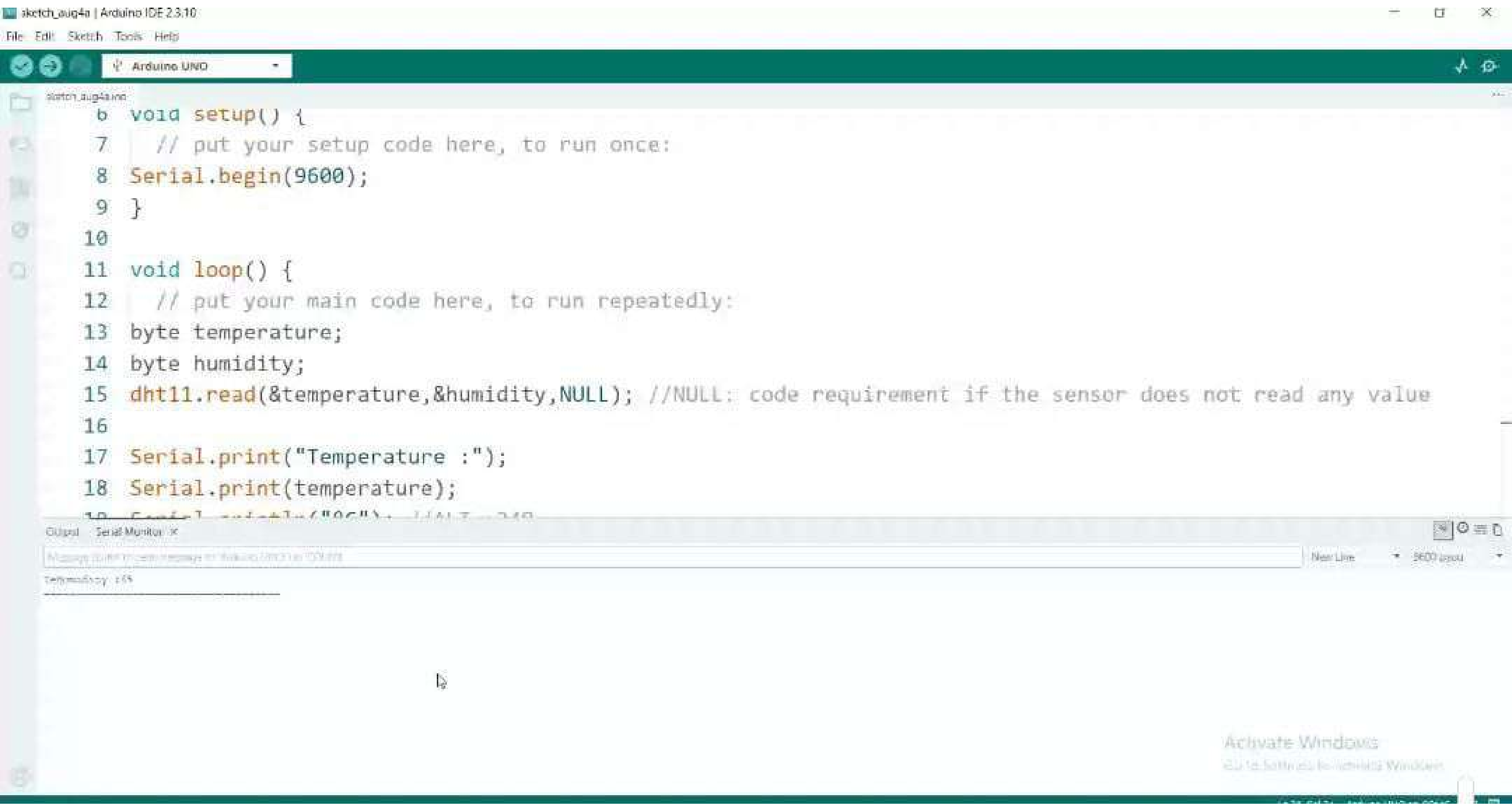

RESET
ICSP2
AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX▶1
RX◀0
DIGITAL (PWM=~)
L
TX
RX
UNO
Arduino
ON
ICSP
1
IOREF
RESET
3V3
5V
GND
GND
VIN
POWER
ANALOG IN
A0
A1
A2
A3
A4
A5
s v g
Keys
fritzing

Board connection



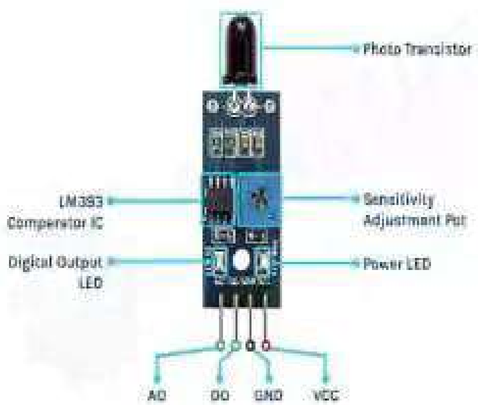
```
#include <SimpleDHT.h>

int pinDHT11=7; //INITIALIZE THE SENSOR PIN
SimpleDHT11 dht11(pinDHT11); // DEFINE THE SENSOR AND ITS PIN

void setup() {
  // put your setup code here, to run once:
Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:
byte temperature;
byte humidity;
dht11.read(&temperature,&humidity,NULL); //NULL: code requirement if the sensor does not read any value


}

```


sketch_aug4a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
Arduino UNO
sketch_aug4a.ino
void setup() {
// put your setup code here, to run once:
Serial.begin(9600);
}
void loop() {
// put your main code here, to run repeatedly:
byte temperature;
byte humidity;
dht11.read(&temperature,&humidity,NULL); //NULL: code requirement if the sensor does not read any value
Serial.print("Temperature :");
Serial.print(temperature);
Output
Serial Monitor
New Line
9600 baud
Activate Windows


sketch_aug4a | Arduino IDE 2.3.10
File Edit Sketch Tools Help
Arduino UNO
void setup() {
// put your setup code here, to run once:
Serial.begin(9600);
}
void loop() {
// put your main code here, to run repeatedly:
byte temperature;
byte humidity;
dht11.read(&temperature,&humidity,NULL); //NULL: code requirement if the sensor does not read any value
Serial.print("Temperature :");
Serial.print(temperature);
Output
Serial Monitor
New Line
Activate Windows

Activity 3: Detecting fire using Flame Sensor

Getting to Know Your Flame Sensor



Usage: These types of sensors are used for short range fire detection and can be used to monitor projects or as a safety precaution to cut devices off / on.

Range: mostly accurate up to about 3 feet.

How it works: The flame sensor is very sensitive to IR wavelength at 760 nm ~ 1100 nm light.

Analog output (A0): Real-time output voltage signal on the thermal resistance.

Digital output (D0): When the temperature reaches a certain threshold, the output high and low signal threshold adjustable via potentiometer.

Pins:

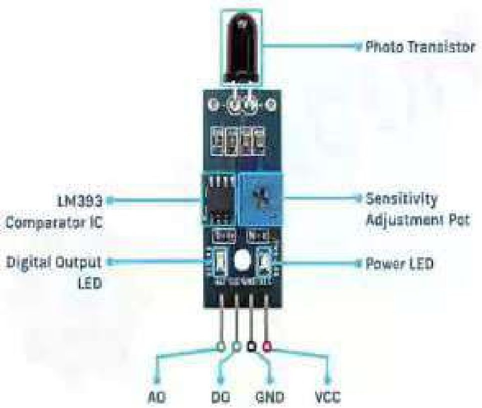
VCC...... Positive voltage input: 5v for analog 3.3v for Digital.

GND............ Ground

A0......... Analog output

D0........ Digital output

Getting to Know Your Flame Sensor



Usage: These types of sensors are used for short range fire detection and can be used to monitor projects or as a safety precaution to cut devices off / on.

Range: mostly accurate up to about 3 feet.

How it works: The flame sensor is very sensitive to IR wavelength at 760 nm ~ 1100 nm light.

Analog output (A0): Real-time output voltage signal on the thermal resistance.

Digital output (D0): When the temperature reaches a certain threshold, the output high and low signal threshold adjustable via potentiometer.

Pins:

VCC...... Positive voltage input: 5v for analog 3.3v for Digital.

GND............. Ground

A0.......... Analog output

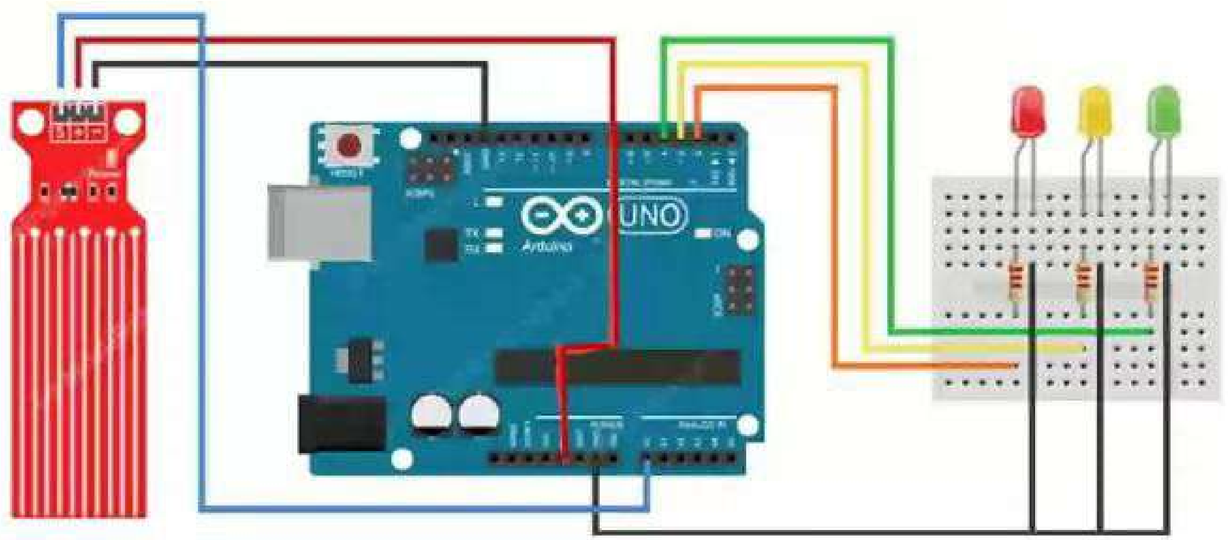
D0......... Digital output

dot.

Activity 3: Detecting fire using Flame Sensor

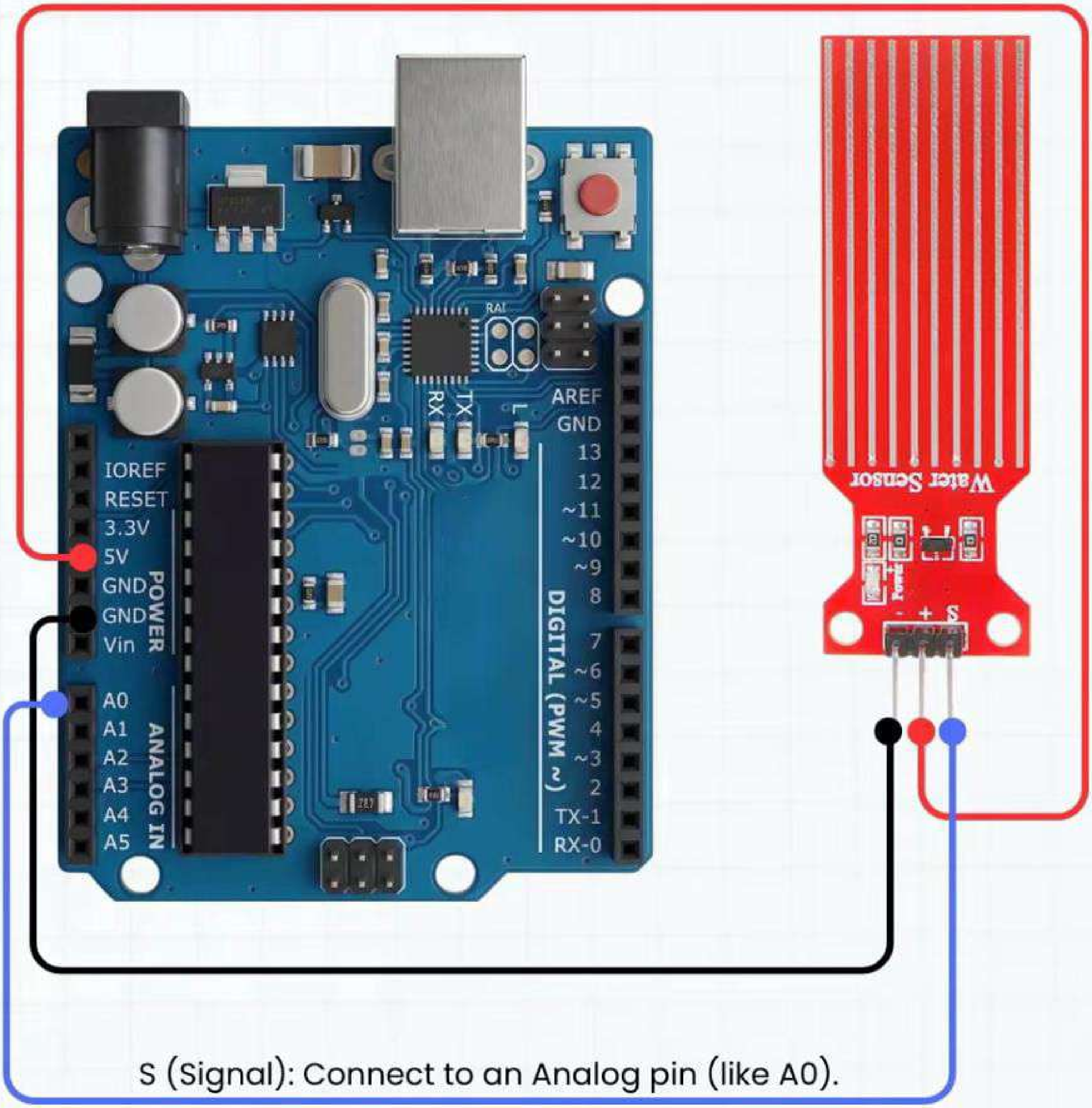
Extra Application 1:

Connection

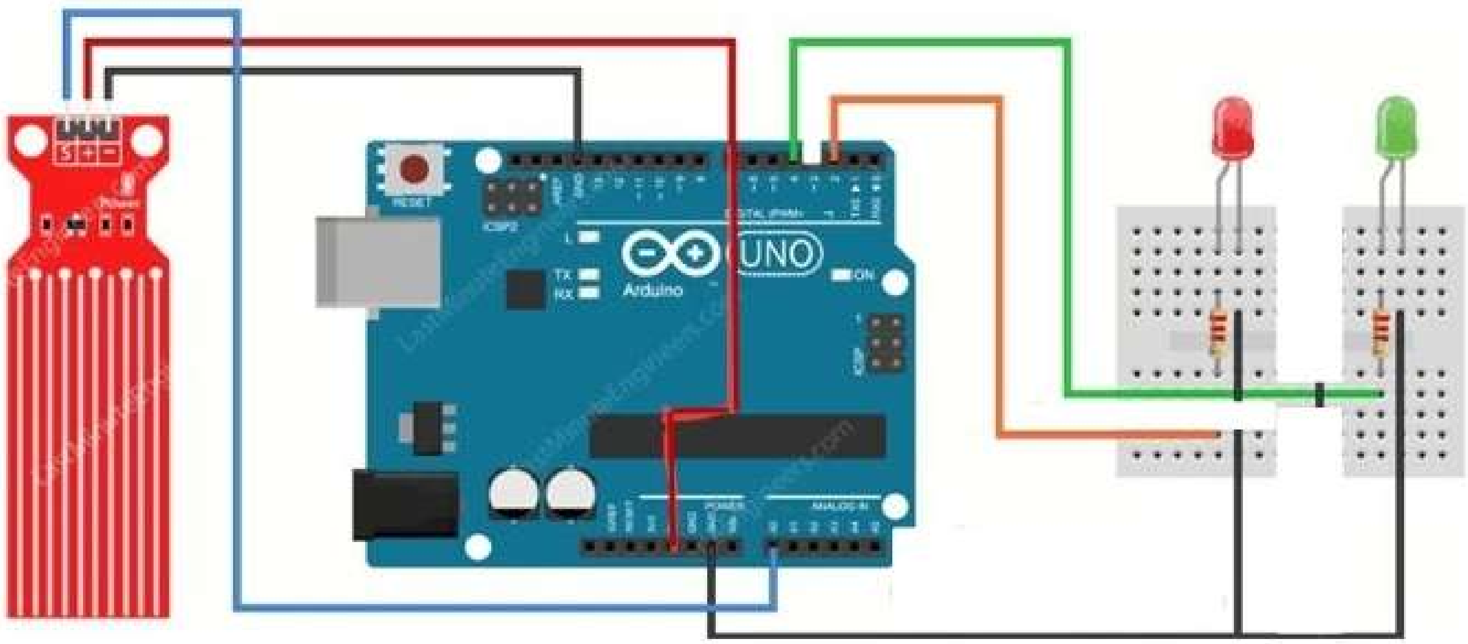


dot

Interfacing with Arduino



S (Signal): Connect to an Analog pin (like A0).



WaterSensor.ino

```
int waterSensor = A0;
int redLed = 8;
int greenLed = 9;
int buzzer=5;

void setup() {

  pinMode(greenLed, OUTPUT);
  pinMode(redLed, OUTPUT);
  Serial.begin(9600);
}

void loop() {

  int waterValue = analogRead(waterSensor);
  Serial.println(waterValue);

  if (waterValue > 450) { //450 can be changed regarding each sensor

    digitalWrite(redLed, HIGH);
    digitalWrite(greenLed, LOW);
    Serial.println("WATER DETECTED");
    tone(buzzer,2000,500);
  }

  else {

    digitalWrite(redLed, LOW);
```

WaterSensor.ino

```
  pinMode(redLed, OUTPUT);
  Serial.begin(9600);
}

void loop() {

  int waterValue = analogRead(waterSensor);
  Serial.println(waterValue);

  if (waterValue > 450) { //450 can be changed regarding each sensor

    digitalWrite(redLed, HIGH);
    digitalWrite(greenLed, LOW);
    Serial.println("WATER DETECTED");
    tone(buzzer,2000,500);
  }

  else {

    digitalWrite(redLed, LOW);
    digitalWrite(greenLed, HIGH);
    Serial.println("NO WATER DETECTED");

  }
  Serial.println("--------------------------------");

  delay(500);
}
```

SoundSensor.ino

```
const int soundAnalogPin = A0;
const int soundDigitalPin = 2;
const int yellowLED = 8;
const int greenLED  = 9;
const int blueLED   = 6;
const int redLED    = 7;
const int buzzerPin = 5;

//should be modified depending the readings of the sensor
const int threshold1 = 54;
const int threshold2 = 58;
const int threshold3 = 62;
const int threshold4 = 68;

void setup() {

  pinMode(yellowLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  pinMode(blueLED, OUTPUT);
  pinMode(redLED, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  pinMode(soundAnalogPin, INPUT);
  pinMode(soundDigitalPin, INPUT);

  Serial.begin(9600);
}

void loop() {

  int soundValue = analogRead(soundAnalogPin);
```

SoundSensor.ino

```
void setup() {

  pinMode(yellowLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  pinMode(blueLED, OUTPUT);
  pinMode(redLED, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  pinMode(soundAnalogPin, INPUT);
  pinMode(soundDigitalPin, INPUT);

  Serial.begin(9600);
}

void loop() {

  int soundValue = analogRead(soundAnalogPin);
  Serial.print("Sound Level: ");
  Serial.println(soundValue);

  digitalWrite(yellowLED, LOW);
  digitalWrite(greenLED, LOW);
  digitalWrite(blueLED, LOW);
  digitalWrite(redLED, LOW);
  noTone(buzzerPin);

  if(soundValue<threshold1){
    digitalWrite(yellowLED, LOW);
    digitalWrite(greenLED, LOW);
    digitalWrite(blueLED, LOW);
    digitalWrite(redLED, LOW);
```

SoundSensor.ino

```
  digitalWrite(redLED, LOW);
  noTone(buzzerPin);

  if(soundValue<threshold1){
    digitalWrite(yellowLED, LOW);
    digitalWrite(greenLED, LOW);
    digitalWrite(blueLED, LOW);
    digitalWrite(redLED, LOW);
    Serial.println("NO SOUND");
  }

  if (soundValue >= threshold1) {
    digitalWrite(yellowLED, HIGH);
    Serial.println("LOW SOUND");
  }
  if (soundValue >= threshold2) {
    digitalWrite(greenLED, HIGH);
    Serial.println("MEDIUM SOUND");
  }
  if (soundValue >= threshold3) {
    digitalWrite(blueLED, HIGH);
    Serial.println("HIGH SOUND");
  }
  if (soundValue >= threshold4) {
    digitalWrite(redLED, HIGH);
    Serial.println("VERY HIGH SOUND!!");
    tone(buzzerPin, 1000);
  }

  delay(1000);
}
```